Replicated Counter Service

1. Requirements under test

A2. Requirement: duplicate suppression by idempotency key, under the same lock; a retry with the same key does not apply the delta again; client has a 2 s deadline, up to 3 retries and one key per operation. Verified by: test_duplicate_key_not_reapplied, test_retry_after_timeout_is_safe, test_request_duplication, test_induced_timeout_with_retry.

A3. Requirement: two clients with 1000 increments each give exactly 2000; shared state is protected by a lock. Verified by: test_concurrent_increments_exact.

B1. Requirement: Lamport clocks (+1 before a local event, max(own, received)+1 on receive), the log format, and time sent in messages. Verified by: captured logs (Appendix A, logs/trace.txt); the L values were checked by hand against the rules. There is no automated test (see section 6).

C1. Requirement: a write is accepted with 2 of 3 acknowledgements and rejected with 1; replicas converge. Verified by: test_majority_commit_two_acks, test_no_commit_below_majority, test_replicas_converge, test_replica_crash_mid_request.

Basic behaviour. Requirement: Increment adds exactly delta; Get on an unknown counter returns found=false. Verified by: test_increment_applies_delta, test_get_missing_counter.

2. Test environment

OS: Windows, run from PowerShell (CPU and RAM were not recorded).

Python: 3.14.7, virtual environment (venv).

Main packages: grpcio 1.84.0, grpcio-tools 1.84.0, protobuf 7.36.2, pytest 9.1.1, pluggy 1.6.0 (full list: Appendix B).

Setup: all processes run on one computer at 127.0.0.1. The tests start servers on free ports, either inside the test process (gRPC server) or as a separate process (subprocess.Popen).

Clean checkout check: git clone, new venv, pip install, python -m pytest -v gives 11 passed in 24.42 s.

3. Test cases

Every test below was run; the actual result matched the expected result and the outcome was PASS in all 11 cases.

T1 test_increment_applies_delta. Purpose: Increment adds exactly delta. Expected: new_value=5, then Get=5. Actual: as expected. Result: PASS.

T2 test_duplicate_key_not_reapplied. Purpose: a repeated key is not applied again. Expected: both replies are 5 (not 10); was_duplicate is False, then True. Actual: as expected. Result: PASS.

T3 test_get_missing_counter. Purpose: unknown counter. Expected: found=False, no exception. Actual: as expected. Result: PASS.

T4 test_concurrent_increments_exact. Purpose: race condition (A3). Expected: 2 threads × 1000 gives exactly 2000. Actual: as expected. Result: PASS.

T5 test_majority_commit_two_acks. Purpose: quorum with one replica stopped. Expected: 2 acknowledgements, new_value=1. Actual: as expected. Result: PASS.

T6 test_no_commit_below_majority. Purpose: below the majority. Expected: two replicas stopped gives fewer than 2 acknowledgements. Actual: as expected. Result: PASS.

T7 test_replicas_converge. Purpose: convergence. Expected: after 10 quorum writes all three replicas hold 10. Actual: as expected. Result: PASS.

T8 test_retry_after_timeout_is_safe. Purpose: timeout and retry. Expected: server slower than the deadline (2500 ms vs 2 s); the retry succeeds and the counter is 1. Actual: as expected. Result: PASS.

F1 test_replica_crash_mid_request. Purpose: replica crash during a write. Expected: the replica process is killed during a write; no exception, at least 2 acknowledgements. Actual: as expected. Result: PASS.

F2 test_request_duplication. Purpose: duplicate request on a real server. Expected: second reply has was_duplicate=True and value 5. Actual: as expected. Result: PASS.

F3 test_induced_timeout_with_retry. Purpose: timeout on a real process. Expected: with --delay-ms 2500, new_value=1 and the counter is 1. Actual: as expected. Result: PASS.

4. Failure-injection results

Replica crash. Injected fault: proc.kill() on replica C during a quorum write. Invariant checked: the write is accepted and no exception reaches the caller. Evidence: F1 PASS and the manual run below.
Duplicate request. Injected fault: the same key is sent twice. Invariant checked: the second reply has was_duplicate=true and the value changed once. Evidence: F2 PASS and the manual run below.
Timeout and retry. Injected fault: flag --delay-ms 2500 with a client deadline of 2 s. Invariant checked: the retry succeeds and the counter grew once. Evidence: F3 and T8 PASS and the logs below.
Below majority. Injected fault: two of three replicas stopped. Invariant checked: the write is not reported as accepted. Evidence: T6 PASS.

Log evidence: timeout and retry (client and a replica started with --delay-ms 2500). The first client SEND (L=1) got no reply. The second SEND (L=2) is the retry with the same key. The replica shows two RECV lines (original and retry) but only one APPLY. After it woke up, the original request found its key in the dictionary and replied without an APPLY (the second SEND IncrementReply).

```
[client-1] SEND Increment(counter=likes:post-42, delta=1) L=1
[client-1] SEND Increment(counter=likes:post-42, delta=1) L=2
[client-1] RECV IncrementReply(new_value=1) L=8 (received L=7)
OK committed value=1 (duplicate: no)
[replica-A] RECV Increment(counter=likes:post-42, delta=1) L=4 (received L=1)
[replica-A] RECV Increment(counter=likes:post-42, delta=1) L=5 (received L=2)
[replica-A] APPLY counter=likes:post-42 -> 1 L=6
[replica-A] SEND IncrementReply(new_value=1) L=7
[replica-A] SEND IncrementReply(new_value=1) L=8
```

Log evidence: duplicate and unavailable replica (manual runs). Repeating the key abc does not change the value. With replica C stopped, the client tries it 4 times (four SEND lines in a row without RECV), but the write is accepted by the other two replicas.

```
OK committed value=10 (duplicate: no)
OK committed value=10 (duplicate: yes)      <- same key abc, value did not change
...
[client-1] SEND Increment(counter=likes:post-42, delta=1) L=14
[client-1] SEND Increment(counter=likes:post-42, delta=1) L=15
[client-1] SEND Increment(counter=likes:post-42, delta=1) L=16
[client-1] SEND Increment(counter=likes:post-42, delta=1) L=17
OK committed value=2 (replicas acked: 2/3, duplicate: no)
```

5. Performance

Method. The script tests/perf_benchmark.py starts three replicas as separate processes. Each configuration runs 2000 Increment calls (with 16 clients, 125 per thread). Every request uses a new uuid4 key. Time is measured with time.perf_counter() and reported in milliseconds. Mean: statistics.mean. p95: sorted(...)[ceil(0.95·n) − 1]. The quorum write calls the three replicas one after another. Each configuration was run once.|

Configuration	Mean latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.33	0.41	2000
Single replica, 16 clients	3.41	4.69	2000

| Quorum (3 replicas), 1 client | 1.09 | 1.32 | 2000 |
| Quorum (3 replicas), 16 clients | 6.52 | 9.21 | 2000 |

Interpretation. With one client, the quorum write is about [X] times slower than a single replica ([A] ms vs [B] ms mean latency), because the client calls the three replicas one after another and waits for each reply. With 16 clients, latency grows in both setups, probably because the server handles at most 8 requests at once and all of them wait for one lock. For a like counter, a mean latency of [N] ms is acceptable when all replicas are healthy, and in return the quorum keeps committed writes if one replica fails. When a replica is down, a write takes seconds because of client retries (0.2, 0.4 and 0.8 s); this case was not measured. All processes run on one computer over localhost, so in a real network each sequential call would add round-trip time.

6. Known limitations

No persistence. Replica state is kept in memory. A restarted replica is empty, and committed writes stored only on crashed replicas are lost.

No catch-up. A replica that fell behind does not catch up by itself. We saw this by hand: after replica C was stopped, it stayed at value 1 while replicas A and B reached 2.

Reads from one replica. Get can return an old value (section 8).

Partly applied failed write. A write acknowledged by only one replica is reported as failed, although that replica has already applied it (section 8).

Limited measurements. One computer, localhost, one run per configuration, no estimate of variation.

Lamport clocks are checked by logs, not by an automated test (section 1, row B1).

7. Event ordering analysis (task B2)

Scenario: client-1 increments x twice, client-2 increments y twice, client-3 only reads (Get). The log of one run is saved in logs/trace.txt, and an excerpt is in Appendix A. Rule: event a happens before event b (a → b) if you can get from a to b using two kinds of steps: the next event in the same process, or send → receive of the same message. If a → b, then L(a) < L(b). The reverse is not true. The diagram is Figure 1 below.

Causal pair 1

client-1 SEND Increment(x) L=1 → replica RECV L=4 (message) → replica APPLY L=8 (order on the replica) → replica SEND IncrementReply L=9 (order on the replica) → client-1 RECV L=10 (message). So the SEND (L=1) happens before the RECV (L=10).

Causal pair 2 (different clients)

client-2 SEND Increment(y) L=1 → replica RECV L=3 (message) → replica APPLY y→1 L=6 → replica SEND GetReply(value=1, found=True) L=11 (order on the replica) → client-3 RECV L=12 (message). Client 3 received the value 1, which exists because of the increment from client 2, although the two clients never exchanged a message directly.

Concurrent pair

client-3 SEND Get(y) L=7 and client-1 SEND Increment(x) L=11. There is no chain in either direction: the replica received Get(y) only at L=10, and the next reply to client 1 was sent at L=18; that reply reached client-1 after its SEND L=11. The fact that 7 < 11 does not mean "earlier". L is a counter of each process. client-1 has a larger value because it had already received the replica's reply with L=9, so it had seen more causal history. It does not mean that the event happened later in real time.

Limit of Lamport clocks

From L values alone you cannot tell whether one event influenced another or the two are independent. Vector clocks can: by comparing two vectors you can say exactly whether one event happened before the other or they are independent.

Figure 1. Happened-before diagram (from logs/trace.txt). Time runs downward along each line; height between different lines means nothing. The number in a circle or square is L; a square is APPLY. Green is chain 1, purple is chain 2, and dashed rings mark the concurrent events 7 and 11.

8. RPC semantics, locking and quorum

RPC semantics (A2)

The design gives at-most-once execution of the change: one logical operation changes the counter at most once, no matter how many times the client retries it. After a timeout the client cannot tell a lost request from a lost reply, so it retries with the *same* idempotency key, created once (uuid4) before the first attempt. The server keeps a dictionary _seen with pairs "key → result". When it sees a key a second time, it returns the stored result with was_duplicate=true and does not add the delta. If the request never arrived, the retry is applied for the first time. If only the reply was lost, the retry gets the same result. In both cases the counter changes once. If all retries fail, the client gets an error and does not know whether the operation was applied, but it is never applied twice.

Protecting shared state (A3)

The server handles requests in a pool of 8 threads (ThreadPoolExecutor(max_workers=8)). The shared state (_values and _seen) is protected by one threading.Lock (self._lock). Checking the key, adding the delta and writing to _seen happen in one critical section. Otherwise two simultaneous retries with the same key could both pass the check. The test test_concurrent_increments_exact (2 threads × 1000) gives exactly 2000. The Lamport clock has its own lock.

Failed but partly applied write (C1)

If only one replica acknowledges, the client reports an error, although that replica has already applied the write. Its value can be seen when reading from that replica, and the other replicas do not have it. A real consensus protocol (for example Raft) prevents this: the write goes through a leader into an ordered log and counts as committed only after it is stored on a majority. Writes that were not committed do not become visible and are overwritten when the leader changes.

Risk of reading from one replica (C1)

Get reads one replica. That replica may have missed a committed write while it was unavailable and can return an old value. Reading from a majority and taking the newest value would reduce this risk.

9. Reflection questions

1. After a timeout, why can the client not tell "request lost" from "reply lost", and how does idempotency make this harmless?

The client only sees that no reply arrived before the deadline. A lost request, a lost reply and a slow server all look the same. A retry with the same key is safe in every case: if the request never arrived it is applied for the first time, and if it did arrive the server returns the stored result without adding again (T8, F3: the counter equals 1).

2. Why does a smaller L on concurrent events not mean it happened first?

L is the counter of one process and shows how much causal history that process has seen. For client-3 L=7 and client-1 L=11 there is no chain of messages, so 7 < 11 says nothing about order. The guarantee goes only one way: a → b gives L(a) < L(b).

3. Which failures does the service tolerate without losing committed writes, and which one breaks the guarantee (safety and liveness)?

One replica failure out of three: a committed write is on at least two replicas, so at least one surviving replica has it (safety), and the two remaining replicas can still accept writes (liveness; T5, F1). Two failures break liveness: there is no majority, so no write is accepted (T6). Because state is kept in memory, losing both replicas that hold a committed write also breaks safety. In addition, reading from one replica can return an old value.

4. What is the smallest change to survive replica restarts, and which week 6-7 mechanism does it prepare?

Write every applied operation (with its key) to a file before replying to the client, and rebuild the state from the file at startup. This prepares the persistent log of Raft. A restarted replica would also need catch-up replication.

Appendix A. Log excerpt from scenario B2 (30 lines; full log: logs/trace.txt)

Clients (first 10 lines):

[client-1] SEND Increment(counter=x, delta=1) L=1

[client-2] SEND Increment(counter=y, delta=1) L=1

[client-3] SEND Get(counter=x) L=1

[client-3] RECV GetReply(value=0, found=False) L=6 (received L=5)

[client-3] SEND Get(counter=y) L=7

[client-2] RECV IncrementReply(new_value=1) L=8 (received L=7)

[client-1] RECV IncrementReply(new_value=1) L=10 (received L=9)

[client-2] SEND Increment(counter=y, delta=1) L=9

[client-1] SEND Increment(counter=x, delta=1) L=11

[client-3] RECV GetReply(value=1, found=True) L=12 (received L=11)

Replica (20 lines):

[replica-A] RECV Get(counter=x) L=2 (received L=1)

[replica-A] SEND GetReply(value=0, found=False) L=5

[replica-A] RECV Increment(counter=y, delta=1) L=3 (received L=1)

[replica-A] RECV Increment(counter=x, delta=1) L=4 (received L=1)

[replica-A] APPLY counter=y -> 1 L=6

[replica-A] SEND IncrementReply(new_value=1) L=7

[replica-A] APPLY counter=x -> 1 L=8

[replica-A] SEND IncrementReply(new_value=1) L=9

[replica-A] RECV Get(counter=y) L=10 (received L=7)

[replica-A] SEND GetReply(value=1, found=True) L=11

[replica-A] RECV Increment(counter=y, delta=1) L=12 (received L=9)

[replica-A] RECV Increment(counter=x, delta=1) L=13 (received L=11)

[replica-A] APPLY counter=y -> 2 L=14

[replica-A] SEND IncrementReply(new_value=2) L=16

[replica-A] RECV Get(counter=x) L=15 (received L=13)

[replica-A] APPLY counter=x -> 2 L=17

[replica-A] SEND IncrementReply(new_value=2) L=18

[replica-A] SEND GetReply(value=2, found=True) L=19

[replica-A] RECV Get(counter=y) L=22 (received L=21)

[replica-A] SEND GetReply(value=2, found=True) L=23

The replica lines were printed by different threads, so they are not always in increasing L order (for example SEND L=16 appears before RECV L=15). The L values themselves were issued by the clock without repeats, so the order of the replica's events is clear from L.

Appendix B. Package versions (from the pip install output of the test environment)

colorama==0.4.6

grpcio==1.84.0

grpcio-tools==1.84.0

iniconfig==2.3.1

packaging==26.3

pip==26.2.1
pluggy==1.6.0
protobuf==7.36.2
pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0